

HMI Tostado — Estado del Proyecto

Panel de control para tostadora de café industrial
Aplicación web responsiva (móvil / tablet / escritorio), desplegada en Vercel.
Fecha: 22 de junio de 2026 · Versión: 0.1.0

1. Resumen del avance

El panel ya está **funcional y desplegado en línea**. Cuenta con **4 pantallas operativas** que permiten monitorear y controlar la tostadora en tiempo real.

Pantalla	Qué permite hacer
Monitoreo	Ver temperatura en vivo, curva de tostado, ficha técnica del lote, control de calor y flujo de aire
Succión	Encender/apagar el motor, ver caudal con medidor animado, ajustar velocidad, monitorear vibración
Energía	Consumo total en vivo, control individual de 4 resistencias eléctricas, diagnóstico
Ajustes	Configurar velocidad de 3 motores y el setpoint térmico, con alertas de rango

Hoy el sistema opera con **datos simulados realistas**. La estructura ya está lista para conectar los sensores reales más adelante sin rehacer la interfaz.

2. Funcionalidades implementadas

2.1 Lectura de temperatura en vivo + tendencia

La pantalla principal muestra la temperatura actual en grande, junto con la **tendencia** (cuántos grados sube o baja por minuto), que cambia de color e ícono según la dirección.

```
<h2>{Math.round(m.temperature)}<span>°C</span></h2>
...
<div className={m.trend >= 0 ? 'text-secondary' : 'text-primary'}>
  <Icon name={m.trend >= 0 ? 'trending_up' : 'trending_down'} />
  <span>{m.trend >= 0 ? '+' : ''}{m.trend}°/min</span>
</div>
```

2.2 Curva de tostado (Temperatura vs Tiempo)

Gráfico en vivo con la curva de tostado. Incluye **líneas de referencia punteadas** para el "Primer Crack" y el "Objetivo final", y un degradado bajo la curva. Si no hay proceso en marcha, invita a iniciar uno.

```
// Línea de referencia para el Primer Crack
datasets.push({
  label: `Primer Crack (${firstCrack}°)\`,
  data: labels.map(() => firstCrack),
  borderColor: '#7ddc7a',
  borderDash: [5, 5], // línea punteada
  pointRadius: 0,
})
```

```
// Mensaje cuando aún no hay datos del proceso
if (data.length === 0) {
  return <div>Sin datos de proceso – pulsa «Iniciar Proceso»</div>
}
```

2.3 Iniciar / detener el proceso de tostado

Un botón "**Iniciar Proceso**" arranca un nuevo tostado: enciende el calor y empieza a registrar la temperatura desde cero, agregando un punto a la curva **cada minuto**.

```
// Arranca un nuevo tostado y reinicia el historial de la curva
const startRoast = () =>
  setState((p) => ({
    ...p,
    roastRunning: true,
    actuators: { ...p.actuators, heat: true },
    tempHistory: [{ minute: 0, temperature: Math.round(p.temperature * 10) / 10 }],
  })))
```

```
// Cada 60 s se agrega un punto a la curva, sólo si el proceso está en curso
setInterval(() => {
  setState((prev) => {
    if (!prev.roastRunning) return prev
    const minute = (last ? last.minute : -1) + 1
    return { ...prev, tempHistory: [...prev.tempHistory, entry].slice(-MAX_HISTORY) }
  })
}, SNAPSHOT_INTERVAL_MS)
```

2.4 Ficha técnica del lote + exportación a CSV / Excel / PNG

Cada lote muestra su ficha (variedad, origen, propietario, kg tostados) y permite **exportar** los datos en tres formatos: **CSV** (datos + historial), **XLSX** (tres hojas, con el **gráfico de la curva embebido como imagen**) y **PNG** (sólo la curva).

```
// Excel con la curva de tostado embebida como imagen
export async function exportFichaXLSX(batch, history, chartImage) {
  const ExcelJS = (await import('exceljs')).default
  const wb = new ExcelJS.Workbook()

  wb.addWorksheet('Ficha Técnica') // datos del lote
  wb.addWorksheet('Historial Temp') // temperatura por minuto

  if (chartImage) {
    const s3 = wb.addWorksheet('Gráfico')
    const imageId = wb.addImage({ base64: chartImage, extension: 'png' })
    s3.addImage(imageId, { tl: { col: 0, row: 1 }, ext: { width: 820, height: 420 } })
  }
}
```

2.5 Control de flujo de aire

Slider de 0 a 100 % más botones de paso (+/- 5 %), con lectura del caudal y las RPM del motor.

```
<StepButton onClick={() => m.setFan(m.airflow.percent - 5)}>-</StepButton>
<Slider value={m.airflow.percent} onChange={m.setFan} />
<StepButton onClick={() => m.setFan(m.airflow.percent + 5)}>+</StepButton>
```

2.6 Sistema de succión con medidor animado

Botón de encendido tipo "power", **medidor (gauge) de velocidad de caudal**, métricas derivadas en vivo (rendimiento, frecuencia, presión) y un indicador de **vibración** que reacciona al motor. El aspa gira más rápido cuanto mayor es la velocidad.

```
// Métricas calculadas a partir de la velocidad real del motor
export function suctionMetrics(speed) {
  return {
    velocity: +(speed * 0.45).toFixed(1), // m/s
    efficiency: Math.round(speed * 0.95), // %
    frequency: +(speed * 0.6).toFixed(1), // Hz
    pressure: +(speed * 0.12).toFixed(1), // hPa
  }
}
```

```
// El aspa gira más rápido a mayor velocidad (se detiene si el motor está apagado)
<div style={{
  animation: s.running ? `spin ${Math.max(0.3, 2 - s.speed / 60)}s linear infinite` : 'none',
  opacity: 0.3 + s.speed / 100,
}} />
```

2.7 Control de energía: 4 resistencias + consumo total

Cada resistencia se enciende/apaga de forma individual mostrando su consumo en kW. El **consumo total** se suma en vivo, y un botón **"Apagar Todo"** corta todas a la vez.

```
// Suma del consumo de todas las resistencias encendidas
export function totalConsumption(resistances) {
  return +resistances.reduce((sum, r) => sum + (r.on ? r.kw : 0), 0).toFixed(1)
}
```

```
// Encender/apagar una resistencia (asigna un consumo realista al encender)
const toggleResistance = (index) =>
  setState((p) => ({
    ...p,
    resistances: p.resistances.map((r, i) =>
      i === index ? { on: !r.on, kw: !r.on ? +(3.1 + Math.random() * 0.2).toFixed(1) : 0 } :
      r),
  }))
```

2.8 Ajustes: motores y setpoint térmico con alertas

Sliders para 3 motores y para el setpoint térmico (0–450 °C). Si el setpoint supera el rango óptimo, aparece una **alerta automática** de consumo elevado.

```
const setpointHigh = m.setpoint > 230
...
<Slider value={m.setpoint} min={0} max={450} onChange={m.setSetpoint} />
{setpointHigh && (
  <Alerta>El setpoint actual supera el rango de eficiencia óptima.</Alerta>
)}
```

2.9 Parada de emergencia

Una sola acción apaga **todo** de forma segura: calor, succión y todas las resistencias.

```
const emergencyStop = () =>
  setState((p) => ({
    ...p,
    emergency: true,
    actuators: { vacio: false, heat: false },
    suction: { ...p.suction, running: false, targetSpeed: 0 },
    resistances: p.resistances.map(() => ({ on: false, kw: 0 })),
  })))
```

3. Estado actual y próximos pasos

Listo hoy:

- 4 pantallas funcionales y navegables, totalmente responsivas.
- Datos simulados realistas en tiempo real (temperatura, succión, resistencias, vibración).
- Curva de tostado en vivo con referencias de Primer Crack y Objetivo.

- Exportación a CSV, Excel (con gráfico) y PNG.
- Controles operativos: calor, flujo de aire, succión, resistencias, motores, setpoint, parada de emergencia.
- Desplegado en línea (Vercel).

Siguiente fase:

- Conectar los **sensores reales** del Arduino para reemplazar los datos simulados (la interfaz no necesita cambios, sólo la fuente de datos).
- Validar lecturas reales contra la simulación.

Tecnologías: React + Vite + Tailwind CSS · Gráficos con Chart.js · Exportación con ExcelJS.